

ClientFlow AI

Phased Implementation Guide

Complete Frontend, Backend & Database Specifications by Phase

Document Type:	Technical Implementation Roadmap
Total Phases:	5 Phases
Timeline:	21 weeks total
Date:	February 14, 2026

Implementation Phases Overview

Phase	Timeline	Key Deliverables
Phase 1: Foundation	Week 1-2	Auth, Database, Basic Job Management
Phase 2: Core Workflow	Week 3-5	Full Pipeline, AI Proposals, Marketing
Phase 3: Intelligence	Week 6-9	Memory, Discovery Agent, Analytics
Phase 4: Communication	Week 10-14	Email, Scheduling, Negotiation Agents
Phase 5: Autonomous	Week 15-21	LangGraph, Preference Learning, Voice

PHASE 1: Foundation & Core Infrastructure

Timeline: Week 1-2 (14 days) | Goal: Working auth + basic job CRUD

Feature 1.1: Authentication System

FRONTEND - Signup Page

Page: /signup

Components:

- SignupForm with email + password inputs
- Email validation (format check)
- Password requirements display (8+ chars, 1 uppercase, 1 number)
- Continue with Google button
- Error messages below each field

User Flow:

1. User enters email → validates format
2. User enters password → shows checkmarks for requirements
3. Click Sign Up → calls Supabase auth
4. Success → redirect to /auth/check-email
5. Error → show message (email exists, weak password, etc)

State Management:

- email (string)
- password (string)
- errors (object)
- loading (boolean)

BACKEND - Auth API

Endpoint: POST /api/auth/setup-profile

Purpose: Create user profile after signup

Process:

1. Get authenticated user from Supabase
2. Check if profile exists
3. If not, INSERT into users table
4. Return success

Alternative: Database trigger (preferred)

- Automatically creates user row when auth.users row created
- No API call needed

Session Management:

- Supabase handles JWT tokens automatically
- Tokens refresh automatically
- Check session with: `supabase.auth.getUser()`

DATABASE - Users Table

Column	Type	Default	Purpose
id	UUID	PK	From auth.users
email	VARCHAR(255)	UNIQUE	User email
plan	VARCHAR(20)	'free'	Subscription tier
ai_credits_used	INTEGER	0	Monthly AI usage
ai_credits_limit	INTEGER	5	Free plan limit
skills	TEXT[]	'{}'	User skills array
created_at	TIMESTAMPTZ	NOW()	Account creation

RLS Policies:

- Users SELECT own profile: `auth.uid() = id`
- Users UPDATE own profile: `auth.uid() = id`

Indexes:

- `idx_users_email` ON `users(email)`

- idx_users_plan ON users(plan)

Feature 1.2: Basic Job Management

FRONTEND - Add Job Page

Page: /app/jobs/new

Form Fields:

1. Title (required, max 500 chars)

- Text input with character counter

2. Platform (required)

- Radio buttons: Upwork | Fiverr | LinkedIn | Other

- Displayed as cards with platform logos

3. Company (optional)

- Text input

4. Job Description (required, min 50 chars)

- Large textarea (10 rows)

- Character counter (0/10000)

- Placeholder explains AI will use this

5. Budget Range (optional)

- Min and Max number inputs

- Currency selector (INR default)

6. Proposed Price (optional)

- Number input

Actions:

- Save Job → saves and goes to jobs list

- Save & Generate → saves and goes to proposal generator

- Cancel → confirms if changes exist

Auto-Save:

- Saves draft to localStorage every 30 seconds

- On page load, asks to restore draft

Validation:

- Title required
- Platform required
- Description min 50 characters
- Budget max $\geq$ budget min

BACKEND - Jobs API

Endpoint: POST /api/jobs

Input Validation:

1. Authenticate user (supabase.auth.getUser)
2. Validate required fields (title, platform, description)
3. Check description length $\geq$ 50 chars
4. Validate budget range
5. Check free plan limit (10 jobs/month)

Process:

1. If free plan, count jobs this month
2. If $\geq$ 10, return 403 with upgrade message
3. INSERT job into database
4. Return job_id and success message

Response:

Success (201): {success: true, job_id: 'uuid', message: 'Job created'}

Error (403): {error: 'Free plan limit reached', upgrade_required: true}

Error (400): {error: 'Missing required fields'}

Error (401): {error: 'Unauthorized'}

DATABASE - Jobs Table

Column	Type	Default	Purpose
id	UUID	gen_random_uuid()	Primary key
user_id	UUID	FK users(id)	Job owner

title	VARCHAR(500)	REQUIRED	Job title
platform	ENUM	REQUIRED	upwork fiverr linkedin other
job_description	TEXT	REQUIRED	Full description
budget_min/max	DECIMAL(10,2)	NULL	Budget range
proposed_price	DECIMAL(10,2)	NULL	Your quote
status	ENUM	'saved'	saved applied replied won lost
followup_date	DATE	NULL	Next follow-up
created_at	TIMESTAMPTZ	NOW()	When saved

Indexes:

- idx_jobs_user_id ON jobs(user_id)
- idx_jobs_status ON jobs(status)
- idx_jobs_followup_date ON jobs(followup_date)

RLS Policies:

- Users SELECT/INSERT/UPDATE/DELETE only own jobs

Triggers:

- Auto-set applied_at when status changes to 'applied'
- Auto-set followup_date to +3 days when status = 'applied'
- Auto-set closed_at when status changes to 'won' or 'lost'

Feature 1.3: Jobs List Page

FRONTEND - Jobs List

Page: /app/jobs

Layout:

- Header with 'Add Job' button (top right)
- Filter tabs: All | Saved | Applied | Replied | Won | Lost
- Search bar (searches title + description)
- Platform filter dropdown
- Sort dropdown (Newest | Oldest | Budget high-low)

Jobs Table Columns:

1. Title (clickable → opens detail modal)
2. Platform (badge with logo)
3. Budget (formatted with currency)
4. Status (color-coded badge)
5. Applied Date (relative: '3 days ago')
6. Follow-Up (date or 'Overdue' in red)
7. Actions (View | Edit | Delete icons)

Job Detail Modal:

- Full job information
- Edit fields inline
- Change status dropdown
- Add/edit notes textarea
- Generate Proposal button (if not done)
- View Proposal button (if exists)
- Save Changes and Close buttons

Empty State:

- Illustration
- 'No jobs yet' message
- 'Add Your First Job' button

Pagination:

- 20 jobs per page
- Page numbers at bottom

BACKEND - Get Jobs API

Endpoint: GET /api/jobs

Query Parameters:

- status (optional): Filter by status
- platform (optional): Filter by platform
- limit (default: 50): Page size
- offset (default: 0): Pagination
- search (optional): Full-text search

Process:

1. Authenticate user
2. Build query with filters
3. Order by created_at DESC
4. Apply pagination
5. Return jobs array + total count

Response:

```
{  
  jobs: [{id, title, platform, status, ...}, ...],  
  total: 42,  
  limit: 50,  
  offset: 0  
}
```

Feature 1.4: Simple Dashboard

FRONTEND - Dashboard Page

Page: /app/dashboard

Stats Cards (top row):

1. Applications This Week

- Large number: Count of jobs where status != 'saved'
- Change vs last week: +3 (green arrow up)

2. Awaiting Reply

- Count of jobs with status = 'applied'
- Average wait time: '4 days avg'

3. Active Conversations

- Count of jobs with status = 'replied'
- 'View all' link

4. Projects Won

- Count of jobs with status = 'won' this month
- Total revenue sum

Quick Actions:

- Add New Job button (prominent, blue)
- Generate Proposal button (secondary)

Recent Activity:

- Timeline of last 10 actions
- Icons for each action type
- Relative timestamps ('2 hours ago')
- Examples:
 - * 'Saved Web Design Project'
 - * 'Applied to Logo Design'
 - * 'Client replied to WordPress Site'

Follow-Ups Due Today:

- List of jobs where followup_date = today
- Each shows: Title, Platform, Days since applied
- 'Send Follow-Up' button
- Empty state: 'No follow-ups due. Great job!'

BACKEND - Dashboard API

Endpoint: GET /api/dashboard/stats

Queries Executed:

1. Count jobs this week (status != 'saved')
2. Count awaiting reply (status = 'applied')
3. Count active (status = 'replied')
4. Count + sum revenue (status = 'won' this month)
5. Get jobs with followup_date = today
6. Get recent 10 activities (created_at or status changes)

Response:

```
{
  applications_this_week: 12,
  awaiting_reply: 8,
  active_conversations: 3,
  projects_won: 2,
  total_revenue: 45000,
  followups_due: [{...}, {...}],
  recent_activity: [{...}, ...]
}
```

Phase 1 Complete Workflow

End-to-End User Flow:

1. User visits landing page (not built yet, placeholder)
2. Clicks 'Start Free'
3. Lands on /signup
4. Enters email + password OR uses Google OAuth
5. Supabase creates auth user
6. Database trigger creates user profile
7. Email verification sent (if email/password)
8. User verifies email (checks inbox, clicks link)
9. Redirected to /app/dashboard
10. Sees empty dashboard
11. Clicks 'Add New Job'
12. Lands on /app/jobs/new
13. Fills form: Title, Platform, Description, Budget
14. Clicks 'Save Job'
15. API validates + inserts into jobs table
16. Redirected to /app/jobs
17. Sees job in table with status 'Saved'
18. Returns to /app/dashboard
19. Sees 'Applications This Week: 1'

At this point:

- User can add unlimited jobs (free plan)
- Can view all jobs in list
- Can search/filter jobs
- Dashboard shows basic stats
- No AI features yet (that's Phase 2)

Phase 1 Testing Checklist

Feature	Test Case	Expected Result
Signup	Valid email + password	Account created, email sent
Signup	Duplicate email	Error: Email already exists
Signup	Weak password	Error shown, form blocked
Login	Correct credentials	Redirect to dashboard
Login	Wrong password	Error: Invalid credentials
Login	Unverified email	Error: Verify email first
Add Job	All required fields	Job saved, redirect to list
Add Job	Missing title	Error: Title required
Add Job	Description < 50 chars	Error: Too short
Jobs List	No jobs	Empty state shown
Jobs List	Filter by platform	Only Upwork jobs shown
Jobs List	Search 'design'	Matching jobs shown
Dashboard	No jobs	All stats show 0
Dashboard	1 saved job	Applications: 0 (saved != applied)

Phase 1 Deliverables Summary

Frontend:

- /signup page with email/password + Google OAuth
- /login page
- /app/dashboard with stats cards
- /app/jobs (list view with filters)
- /app/jobs/new (add job form)
- Protected route middleware
- Responsive design (mobile + desktop)

Backend:

- POST /api/auth/setup-profile

- POST /api/jobs (create job)
- GET /api/jobs (list with filters)
- GET /api/dashboard/stats
- Session validation middleware

Database:

- users table with RLS
- jobs table with RLS
- Indexes on user_id, status, created_at
- Database triggers for auto-timestamps
- Test data seeding script

Ready for Phase 2:

- User can signup/login
- User can add and track jobs
- Foundation for AI proposal generation